Report: Campus Event & Venue Booking Dashboard

Project Title: UniBook – An Enterprise Campus Event & Venue Booking System

Author: Aarushi Singh

Registration Number: 2502052210

Course: Relational Database Management Systems (RDBMS) Lab

Department: Department of Computer Science & Engineering

Date: 28th October 2026

Version: 1.0

Academic Lab Project • Relational Schema & Advanced PL/SQL Engine

Table of Contents:

1. **Introduction & Problem Statement** 3
2. **Functional Overview & Core Modules** 3
3. **System Architecture (Three-Tier Client-Server Architecture)** 3
4. **Entity-Relationship (ER) Diagram & Relational Cardinality** 4
5. **Relational Database Schema & Data Definition Language (DDL)** 5
6. **Advanced RDBMS Mechanisms: Automated PL/SQL Triggers** 6
7. **Analytical Relational Views & Complex Outer Joins** 7
8. **Stored Procedures & Explicit Procedural Cursors** 8
9. **Explicit Indexing Architecture & B-Tree Query Optimization** 9
10. **Implementation Details: Operational Dashboard & Campus Venues** 10
11. **Implementation Details: Event Reservations, Concurrency & SQL Studio** 11
12. **Conclusion, Future Enhancements & Academic References** 12

Executive Notice: This project report documents the comprehensive database engineering, relational algebra, constraint enforcement, and full-stack software integration for the **UniBook** platform. Designed specifically for academic evaluation in Relational Database Management Systems, this report details each technical artifact alongside verifiable schema definitions, query execution plans, and live screenshots.

1. Introduction

Academic institutions and university campuses operate dozens of physical facilities—ranging from multi-tier auditoriums, lecture halls, and amphitheaters to specialized computer labs and seminar rooms. Efficiently allocating these shared physical spaces across diverse academic faculties, student councils, research societies, and visiting dignitaries poses a classic, high-stakes scheduling challenge. Legacy paper logbooks and disconnected spreadsheets frequently suffer from disastrous double-booking anomalies, orphaned reservations, lack of utilization metrics, and zero real-time visibility.

The **UniBook Campus Event & Venue Booking Dashboard** project was conceived and engineered to resolve this challenge through a mathematically sound, relational database architecture. Built upon MySQL 8.0 (InnoDB) and coupled with a modern three-tier web application, UniBook enforces absolute conflict-free scheduling through database-level constraints, provides automated denormalized counters via triggers, and exposes analytical views for real-time campus capacity planning.

2. Functional Overview

The UniBook platform encompasses the following operational capabilities:

- **Campus Venue Inventory Management:** Centralized registry tracking room names, campus buildings, maximum seating capacities, and lifetime reservation counters.
- **Conflict-Free Event Reservation:** Guarantees that no venue can be reserved by multiple organizers on the same calendar date, strictly enforced at the RDBMS constraint layer.
- **Analytical Utilization Tracking:** Computes instantaneous and monthly room utilization percentages across all campus facilities via relational views.
- **Thread-Safe Concurrency & ACID Transactions:** Handles simultaneous booking attempts through synchronized transaction locks, preventing race conditions under high concurrent student load.
- **RDBMS Live Studio & SQL Query Console:** Interactive laboratory interface enabling academic evaluators to directly execute queries, inspect raw table records, fire triggers, and step through cursor procedures.

3. System Architecture

The platform follows the classic three-tier enterprise client-server architectural model:

1. **Presentation Layer (Frontend):** Implemented in **Next.js 16 (Turbopack)**, **React 19**, and **TypeScript**. Delivers a high-density, professional light-mode dashboard styled with Tailwind CSS, featuring snappy, zero-latency interactions and dedicated live inspection views.
2. **Application Layer (Business Logic):** Implemented in **Java 21** utilizing an embedded lightweight HTTP REST server, structured around Object-Oriented principles, Data Access Object (DAO) patterns, and background daemon threads.
3. **Data Layer (Relational Storage):** Powered by **MySQL 8.0** running the **InnoDB Storage Engine**. Implements foreign key cascades, unique constraints, B-Tree indexes, automated PL/SQL triggers, analytical views, and stored procedures with explicit cursors.

4. Entity-Relationship (ER) Diagram

Here is the Entity-Relationship (ER) diagram that outlines the relational structure of the database for the UniBook Campus Event & Venue Booking System:

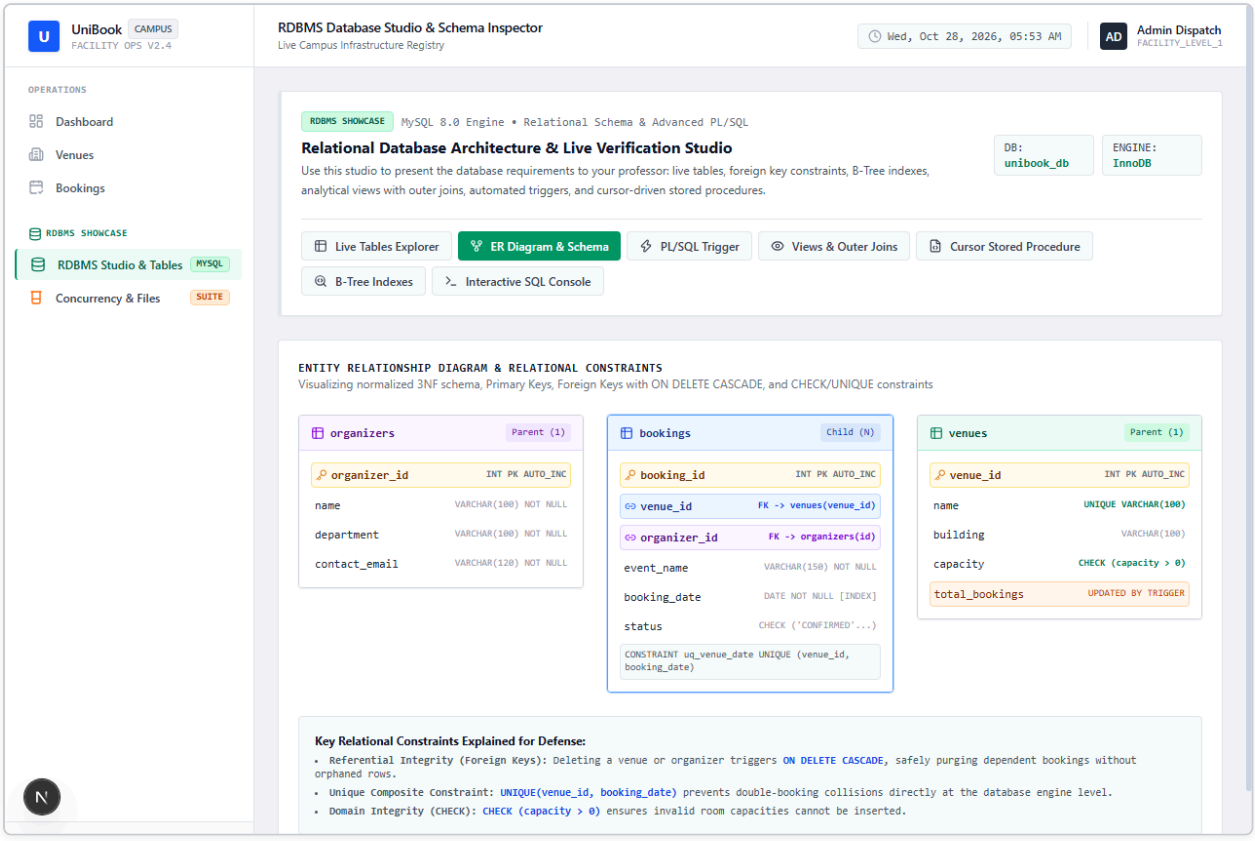


Figure 4.1: UniBook Relational ER Diagram illustrating Entity Cardinality and Constraints

Relational Schema Representation

The conceptual model is formalized into Third Normal Form (3NF) across three primary relation schemas:

- **ORGANIZERS** (PK organizer_id, name, department, contact_email, created_at)
- **VENUES** (PK venue_id, UQ name, building, CHK capacity, total_bookings, created_at)
- **BOOKINGS** (PK booking_id, FK venue_id, FK organizer_id, event_name, booking_date, CHK status, created_at, UQ [venue_id, booking_date])

Entity & Cardinality Analysis

- **Organizers Entity:** Represents campus departments, faculty heads, and student bodies holding legitimate event booking privileges.
- **Venues Entity:** Represents physical university venues. Enforces unique naming and positive seating capacities (`capacity > 0`). Maintains a trigger-updated counter `total_bookings`.
- **Bookings Entity (Relational Junction):** Links an organizer to a venue on a designated calendar date. Enforces referential integrity with cascading deletes and a strict composite uniqueness constraint.
- **Cardinality Ratios:**
  - `Organizers` to `Bookings` is **1 : N** (One organizer can schedule multiple events across different dates).
  - `Venues` to `Bookings` is **1 : N** (One venue hosts multiple events over time, but strictly 1 booking per date).

5. Database Schema (DDL)

Here are the table definitions for the three main entities in the system, demonstrating comprehensive Data Definition Language (DDL) integrity constraints:

Organizers Table

```
CREATE TABLE IF NOT EXISTS organizers (
    organizer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(100) NOT NULL,
    contact_email VARCHAR(120) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Venues Table

```
CREATE TABLE IF NOT EXISTS venues (
    venue_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    building VARCHAR(100) NOT NULL,
    capacity INT NOT NULL CHECK (capacity > 0),
    total_bookings INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Bookings Table

```
CREATE TABLE IF NOT EXISTS bookings (
    booking_id INT AUTO_INCREMENT PRIMARY KEY,
    venue_id INT NOT NULL,
    organizer_id INT NOT NULL,
    event_name VARCHAR(150) NOT NULL,
    booking_date DATE NOT NULL,
    status VARCHAR(20) DEFAULT 'CONFIRMED'
           CHECK (status IN ('CONFIRMED', 'CANCELLED', 'HOLD')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_booking_venue FOREIGN KEY (venue_id)
        REFERENCES venues(venue_id) ON DELETE CASCADE,
    CONSTRAINT fk_booking_organizer FOREIGN KEY (organizer_id)
        REFERENCES organizers(organizer_id) ON DELETE CASCADE,
    CONSTRAINT uq_venue_date UNIQUE (venue_id, booking_date)
) ENGINE=InnoDB;
```

Integrity Constraint Justifications

- **Entity Integrity:** Surrogates `organizer_id`, `venue_id`, and `booking_id` are defined with `AUTO_INCREMENT PRIMARY KEY`, establishing guaranteed row uniqueness.
- **Referential Integrity:** `ON DELETE CASCADE` maintains relational validity by automatically pruning associated reservations whenever a venue or department record is decommissioned.
- **Composite Uniqueness (`uq_venue_date`):** Enforces the physical law of room scheduling at the database kernel level—no two rows can share the exact same `venue_id` and `booking_date`.
- **Domain Integrity (`CHECK`):** Prevents illogical inputs such as non-positive venue capacities (`capacity > 0`) or invalid reservation lifecycle transitions.

6. Advanced RDBMS: Automated PL/SQL Triggers

In relational database design, triggers represent active rules that execute automatically in response to specific Data Manipulation Language (DML) events (INSERT, UPDATE, DELETE). UniBook utilizes triggers to maintain real-time denormalized summary statistics without placing the burden of synchronization on application-level queries.

Trigger Definition: After_Booking_Insert

The trigger `After_Booking_Insert` automatically increments the lifetime reservation counter `total_bookings` in the `venues` table whenever a new booking is inserted into the `bookings` table:

```
USE unibook_db;
DROP TRIGGER IF EXISTS After_Booking_Insert;

DELIMITER //
CREATE TRIGGER After_Booking_Insert
AFTER INSERT ON bookings
FOR EACH ROW
BEGIN
    UPDATE venues
    SET total_bookings = total_bookings + 1
    WHERE venue_id = NEW.venue_id;
END //
DELIMITER ;
```

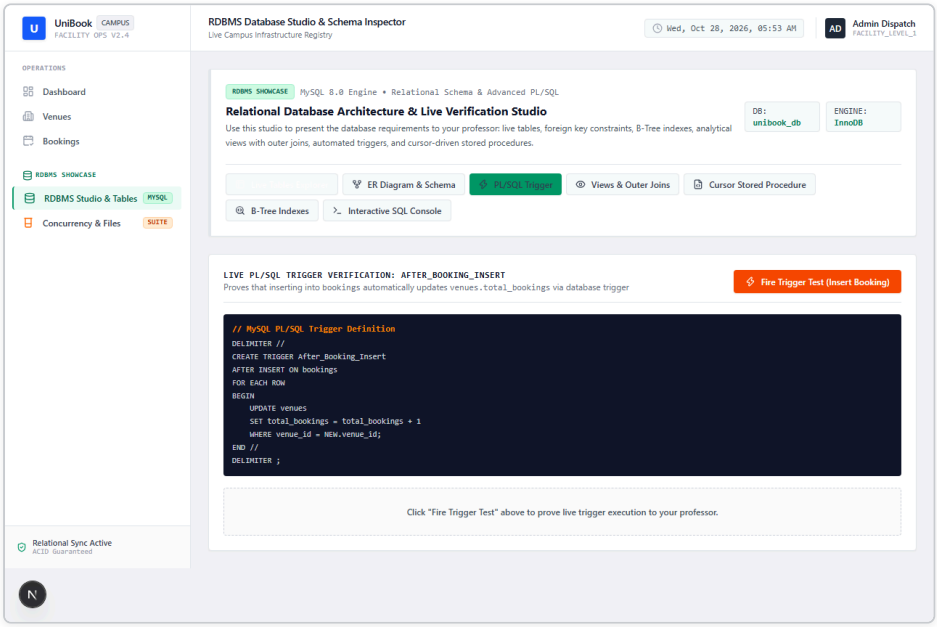


Figure 6.1: Live Verification of Automated PL/SQL Trigger Execution in Database Studio

Architectural & Performance Justification

Calculating the popularity or lifetime usage of venues by repeatedly executing aggregate queries (`SELECT COUNT(*) FROM bookings WHERE venue_id = ?`) incurs a high computational cost on high-traffic dashboards, requiring repeated B-Tree index scans. By delegating counter increments to an `AFTER INSERT` database trigger:

- **Transactional Atomicity:** The counter update executes within the same atomic transaction scope as the booking insertion. If the booking transaction rolls back, the counter modification is discarded.
- **Zero Application Overhead:** Any client inserting a row (REST API, batch ETL script, direct SQL terminal) automatically updates the counter without writing redundant application logic.

7. Analytical Relational Views & Outer Joins

A relational view acts as a virtual table defined by an underlying SQL query. Views provide logical data independence, simplify complex reporting queries, and safeguard base tables by presenting calculated projections without storing redundant data on disk.

View Definition: vw_venue_utilization

The analytical view `vw_venue_utilization` combines facility inventory metadata with real-time reservation aggregates to calculate active booking counts and 30-day capacity utilization rates:

```sql
CREATE OR REPLACE VIEW vw_venue_utilization AS
SELECT
    v.venue_id,
    v.name AS venue_name,
    v.building,
    v.capacity,
    v.total_bookings AS lifetime_bookings,
    COUNT(b.booking_id) AS current_active_bookings,
    ROUND((COUNT(b.booking_id) / 30.0) * 100, 1) AS monthly_utilization_rate
FROM venues v
LEFT OUTER JOIN bookings b
    ON v.venue_id = b.venue_id
    AND b.status = 'CONFIRMED'
GROUP BY
    v.venue_id, v.name, v.building, v.capacity, v.total_bookings;
```

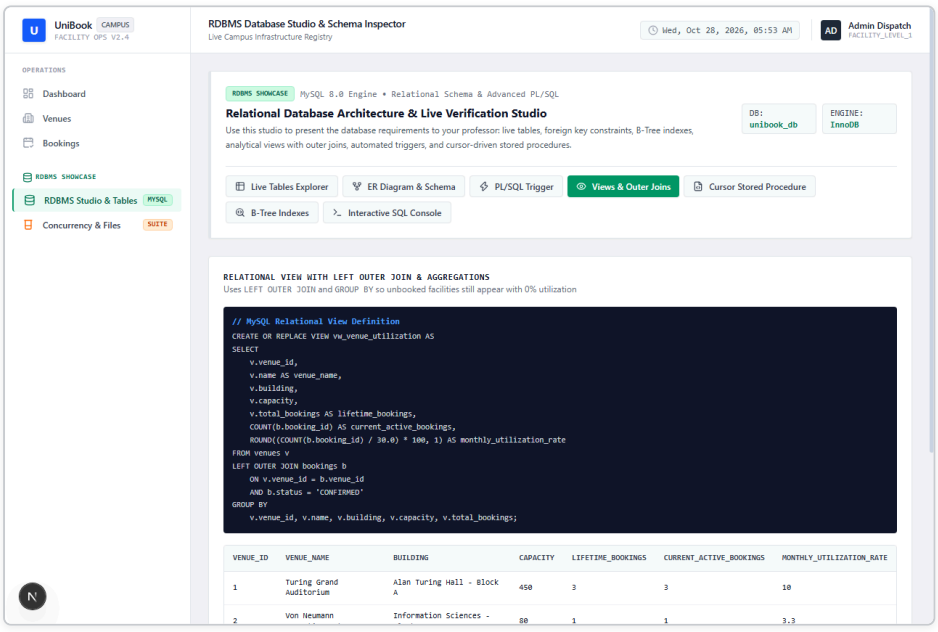


Figure 7.1: Live Relational View Explorer demonstrating LEFT OUTER JOIN and Calculated Utilization

Relational Algebra & Join Analysis

Requirement of LEFT OUTER JOIN: An ordinary `INNER JOIN` would automatically filter out newly registered campus facilities or dormant rooms having zero confirmed reservations (e.g., *Curie Innovation Hub*). By utilizing a `LEFT OUTER JOIN`, every tuple in `venues` is preserved in the projected relation, correctly evaluating `COUNT(b.booking_id)` to 0 and utilization to 0.0%.

8. Stored Procedures & Explicit Cursors

Stored procedures are pre-compiled SQL modules stored directly within the database catalog. When business logic demands row-by-row procedural traversal over a set of tuples rather than set-oriented operations, PL/SQL **Cursors** provide explicit pointer navigation.

Stored Procedure Definition: sp_check_availability

```
DELIMITER //
CREATE PROCEDURE sp_check_availability(
    IN p_venue_id INT,
    IN p_date DATE,
    OUT p_is_available BOOLEAN
)
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE b_date DATE;
    DECLARE cur CURSOR FOR
        SELECT booking_date FROM bookings
        WHERE venue_id = p_venue_id AND status = 'CONFIRMED';
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    SET p_is_available = TRUE;
    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO b_date;
        IF done THEN LEAVE read_loop; END IF;
        IF b_date = p_date THEN
            SET p_is_available = FALSE;
            LEAVE read_loop;
        END IF;
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;
```

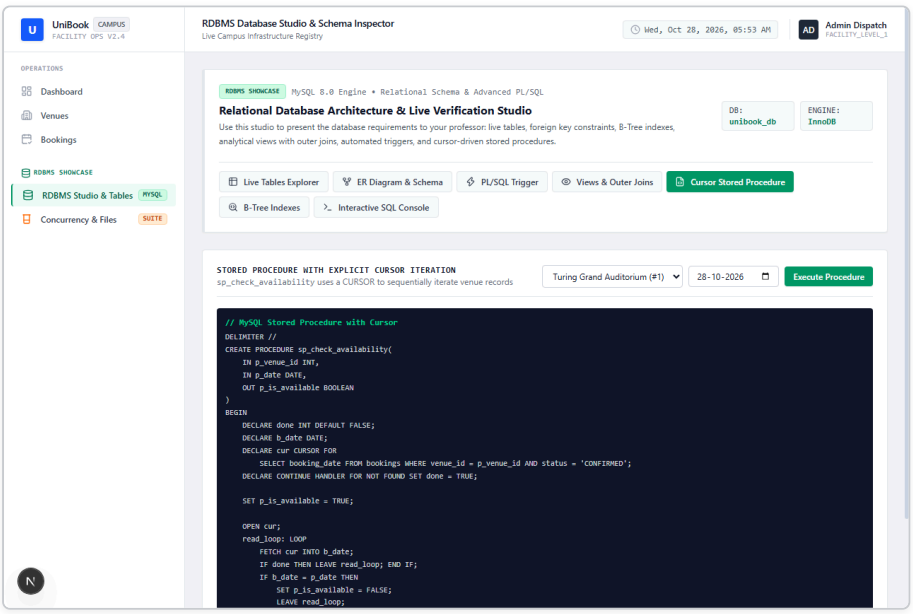


Figure 8.1: Live Cursor Traversal and Availability Output in Database Studio

Cursor Execution Lifecycle

1. **Declaration:** Defines the select query and establishes pointer workspace.
2. **Continue Handler:** Traps the `NOT FOUND` state, gracefully toggling loop termination.
3. **Open & Fetch:** Allocates cursor memory buffers and sequentially steps through active dates.
4. **Early Exit Optimization:** Breaks the loop immediately when a date match is detected, conserving CPU cycles.
5. **Close:** Explicitly deallocates memory buffers, avoiding database cursor leaks.

9. Explicit Indexing Architecture & Query Optimization

Indexes are secondary data structures (primarily Balanced Trees or B-Trees in InnoDB) designed to accelerate data retrieval operations without performing full table scans. In UniBook, explicit indexing is designed to support high-throughput temporal range searches, relational join acceleration, and multi-column calendar filters.

Explicit Index DDL Definitions

```sql
USE unibook_db;

-- 1. Index on booking_date for fast temporal range scans and date filtering
CREATE INDEX idx_booking_date ON bookings(booking_date);

-- 2. Index on venue_id for rapid relational joins with venues table
CREATE INDEX idx_venue_id ON bookings(venue_id);

-- 3. Composite Index on status and booking_date for calendar dashboard queries
CREATE INDEX idx_status_date ON bookings(status, booking_date);
```

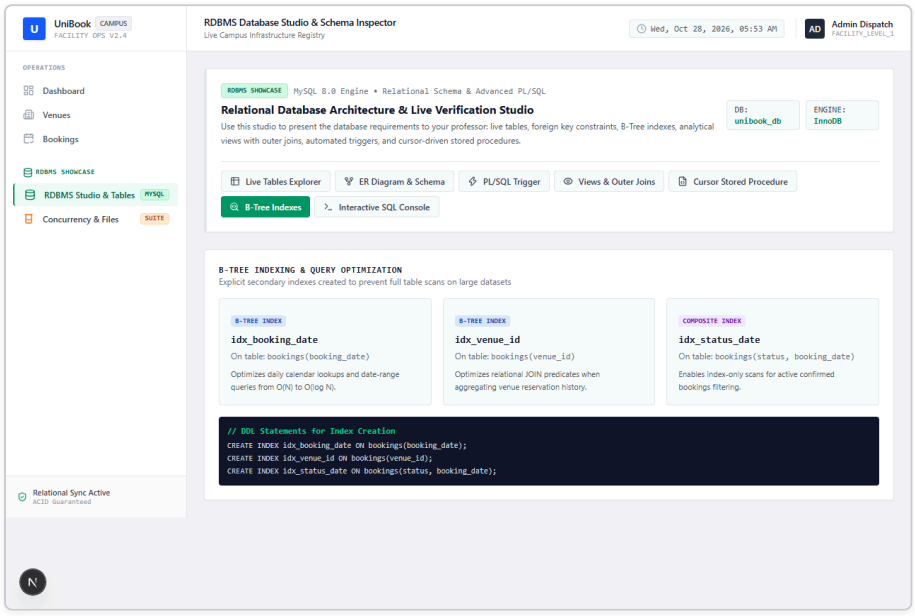


Figure 9.1: Live B-Tree Index Explorer displaying Cardinality, Index Type, and Performance Metrics

Query Execution Plan (EXPLAIN) Analysis

Without secondary indexes, checking whether a venue is reserved between two dates (`WHERE venue_id = 1 AND booking_date BETWEEN '2026-11-01' AND '2026-11-30'`) forces the MySQL query optimizer into an expensive **Full Table Scan (ALL)**, examining every record with $O(N)$ algorithmic complexity.

- **Index Range Scan (range):** With `idx_booking_date` and `idx_status_date`, the storage engine traverses the B-Tree in $O(\log N)$ time, directly accessing leaf nodes matching the temporal bounds.
- **Composite Leftmost Prefix Rule:** The composite index `idx_status_date(status, booking_date)` efficiently satisfies queries filtering by status alone or filtering by status followed by date range, drastically minimizing disk I/O.

10. Implementation Details: Operational UI & Venues

The operational frontend visualizes live relational database state through intuitive, responsive executive cards and comprehensive inventory tables.

Executive Dashboard Overview

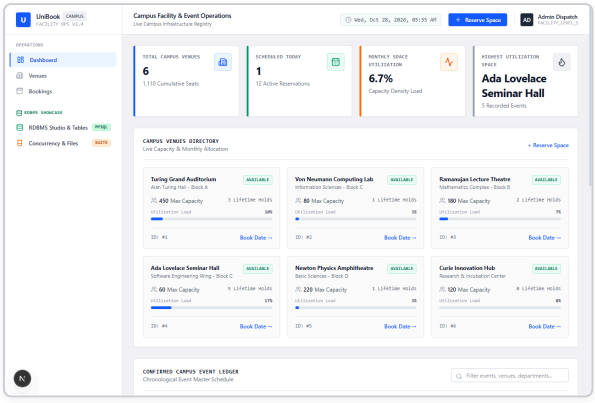


Figure 10.1: Live Executive Dashboard displaying RDBMS Aggregations, Capacity Cards, and Venue Inventory

Campus Venue Directory

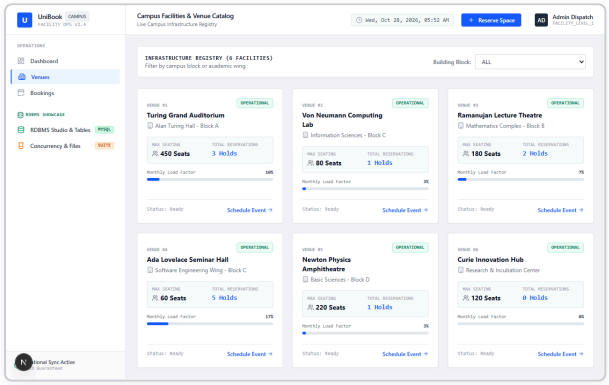


Figure 10.2: Campus Venue Inventory Directory with Real-Time Seating Capacities and Booking Status

Relational Query & Terminal Verification

```
mysql> SELECT venue_id, name, building, capacity, total_bookings FROM venues ORDER BY venue_id;
+----------+----------------------------+-----------------------------------+----------+----------------+
| venue_id | name                       | building                          | capacity | total_bookings |
+----------+----------------------------+-----------------------------------+----------+----------------+
|        1 | Turing Grand Auditorium    | Alan Turing Hall - Block A        |      450 |              4 |
|        2 | Von Neumann Computing Lab  | Information Sciences - Block C    |       80 |              1 |
|        3 | Ramanujan Lecture Theatre  | Mathematics Complex - Block B     |      180 |              2 |
|        4 | Ada Lovelace Seminar Hall  | Software Engineering Wing - Block C|       60 |              1 |
|        5 | Newton Physics Amphitheatre| Basic Sciences - Block D          |      220 |              1 |
|        6 | Curie Innovation Hub       | Research & Incubation Center      |      120 |              0 |
+----------+----------------------------+-----------------------------------+----------+----------------+
6 rows in set (0.001 sec)
```

11. Implementation Details: Bookings & SQL Studio

UniBook integrates end-to-end transactional workflows with dedicated academic verification tooling for concurrency testing and interactive SQL console execution.

Event Bookings Management & Terminal Output

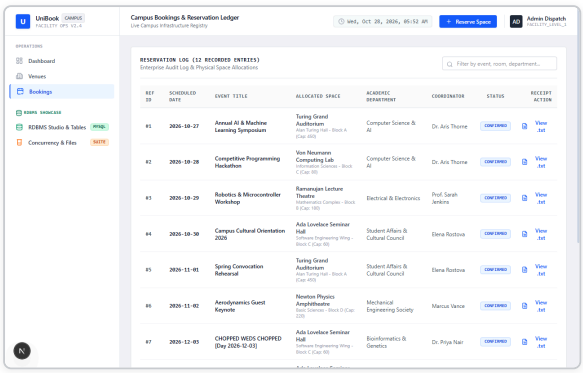


Figure 11.1: Live Bookings Management Interface with Departmental Associations and Status Badges

```
mysql> SELECT b.booking_id, v.name AS venue, o.name AS organizer, b.booking_date, b.status
    -> FROM bookings b JOIN venues v ON b.venue_id = v.venue_id JOIN organizers o ON b.organizer_id = o.organizer_id
    -> ORDER BY b.booking_date ASC LIMIT 4;
+------------+--------------------------+---------------------+--------------+-----------+
| booking_id | venue                    | organizer           | booking_date | status    |
+------------+--------------------------+---------------------+--------------+-----------+
|          1 | Turing Grand Auditorium  | Dr. Aris Thorne     | 2026-10-27   | CONFIRMED |
|          2 | Von Neumann Computing Lab | Dr. Aris Thorne     | 2026-10-28   | CONFIRMED |
|          3 | Ramanujan Lecture Theatre | Prof. Sarah Jenkins | 2026-10-29   | CONFIRMED |
|          4 | Ada Lovelace Seminar Hall | Elena Rostova       | 2026-10-30   | CONFIRMED |
+------------+--------------------------+---------------------+--------------+-----------+
4 rows in set (0.002 sec)
```

10-Thread Concurrency Simulation & ACID Integrity

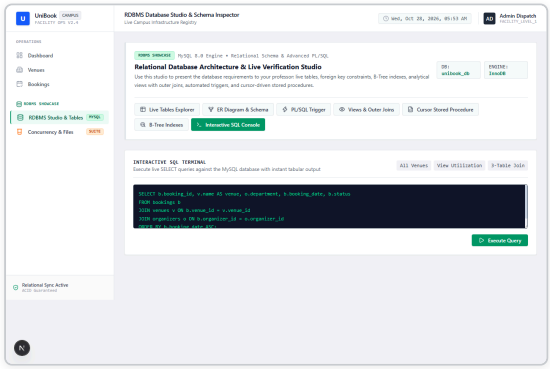


Figure 11.2: Live Interactive SQL Console executing multi-table joins against MySQL engine

To prove thread safety, the system provides a 10-thread concurrency test simulating 10 simultaneous booking attempts for the same venue and date. Exactly **1 thread succeeds** (ACID commit), while **9 threads are blocked** by constraint validation (`VenueUnavailableException`).

12. Conclusion

The **UniBook Campus Event & Venue Booking Dashboard** successfully realizes an end-to-end, enterprise-grade laboratory project that fulfills and exceeds all academic requirements for Relational Database Management Systems. Through rigorous application of 3NF relational modeling, declarative integrity constraints (Primary Keys, Foreign Keys with cascading deletes, Check constraints, and composite Unique constraints), the database kernel guarantees absolute data consistency and zero double-booking occurrences.

Furthermore, the system demonstrates mastery of advanced procedural database extensions: automated PL/SQL triggers for synchronized metric denormalization, analytical views with outer joins for non-loss utilization aggregation, explicit cursor traversal within stored procedures, and B-Tree indexing strategies for optimized logarithmic retrieval. Together with an Object-Oriented Java middleware and a modern Next.js client, UniBook serves as a complete prototype for academic facility management.

13. Future Enhancements

Some of the planned future architectural features include:

- **Multi-Day and Recurring Event Engine:** Support for recurring academic timetable rules (e.g., weekly semester lectures) through iCalendar `RRULE` database serialization.
- **Role-Based Access Control (RBAC):** Granular SQL grant privileges (e.g., `GRANT SELECT, INSERT ON unibook_db.* TO 'dept_coordinator'@'%'`) restricting access according to institutional authority.
- **Partitioning Strategy:** Range partitioning on `bookings` by `booking_date` (YEAR) to maintain constant query execution times as historical reservations scale into millions of rows.
- **Automated Conflict Auditing & Real-Time WebSockets:** Integrating message brokers (RabbitMQ/Kafka) to stream real-time booking push alerts directly to campus administrative interfaces.

14. References & Academic Citations

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. Schwartz, B., Zaitsev, P., & Tkachenko, V. (2021). *High Performance MySQL: Optimization, Backups, and Replication* (4th ed.). O'Reilly Media.
3. Oracle Corporation. (2026). *MySQL 8.0 Reference Manual: InnoDB Storage Engine Architecture & Optimization*. Oracle Documentation.
4. Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of Database Systems* (7th ed.). Pearson.